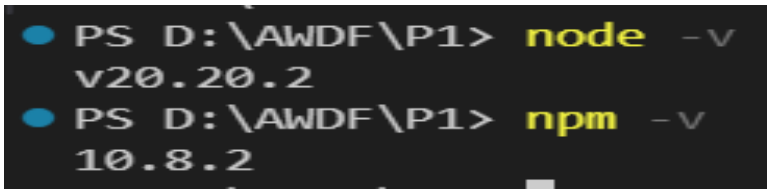
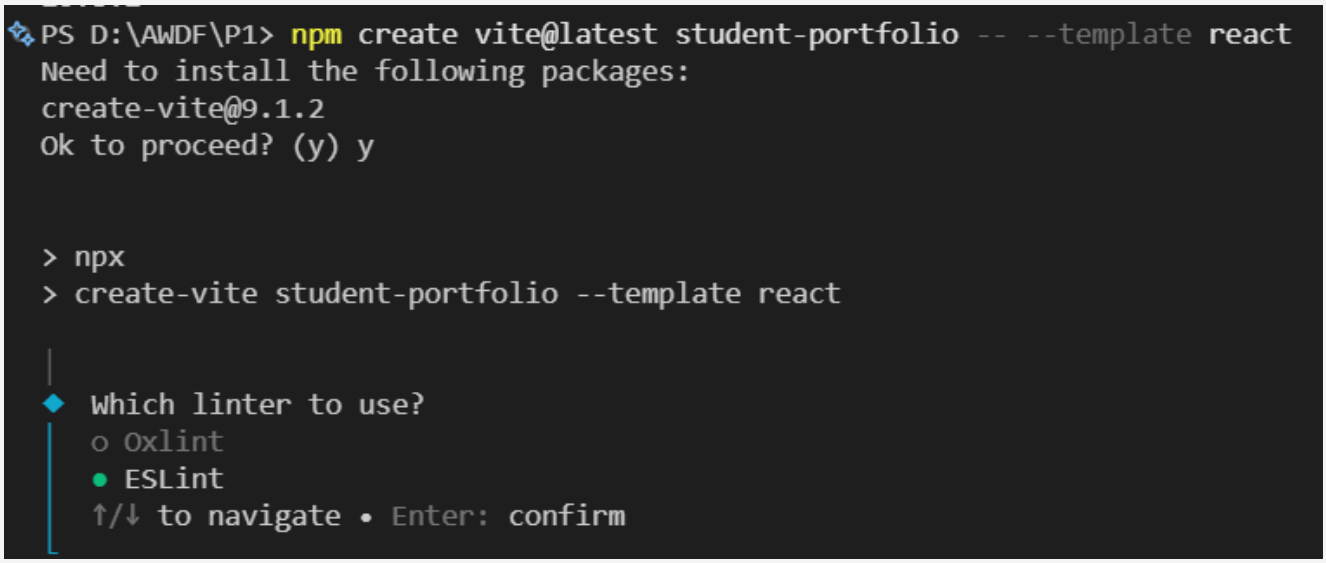
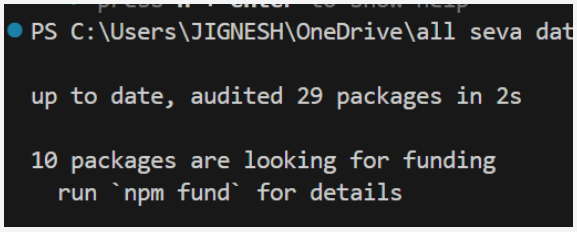
Practical 1: Introduction to React and Component Architecture

Objective	To set up a React development environment using Vite and build a static UI using independently structured, reusable components.
Problem Statement	• Create a React application using Vite for a student portfolio page. • The application must include at least 4 reusable components: Header, About, Skills, and Footer. • Each component must be independently structured and composed into a single-page layout. • No logic or JSX should be duplicated across components. • Props must be used to pass at least one piece of data into a minimum of 2 components.

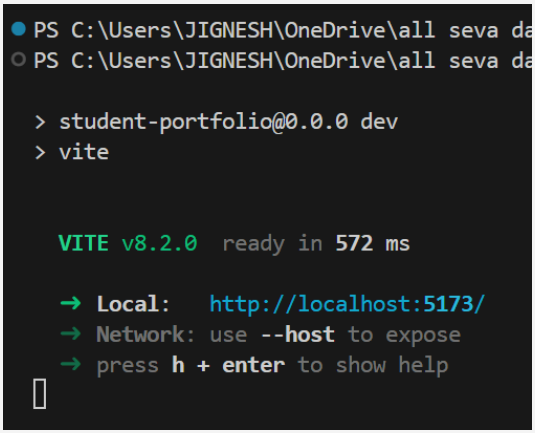
Steps	1. Verify Node.js and npm are installed:
	`node -v`
	`npm –v`
	 
	2. Scaffold the React app using Vite:
	`npm create vite@latest student-portfolio -- --template react`
	 

cd student-portfolio

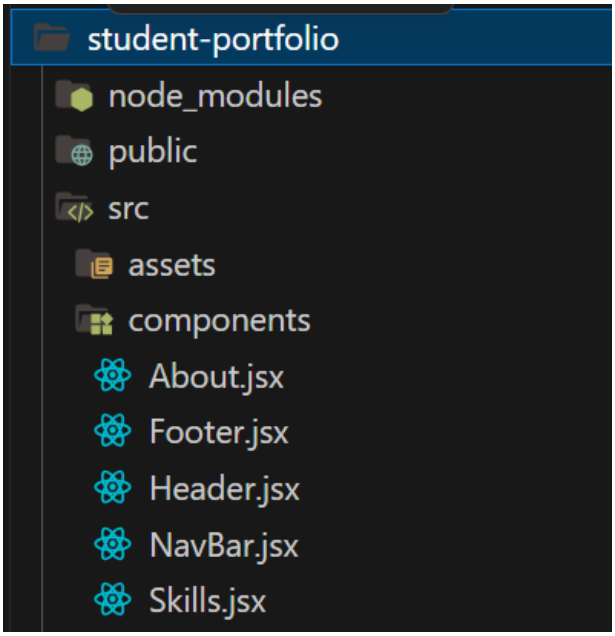
npm install



npm run dev



3. Inside src/components, create four files: Header.jsx, About.jsx, Skills.jsx, Footer.jsx.



4. Build the Skills component to accept a props array and render it as a list:

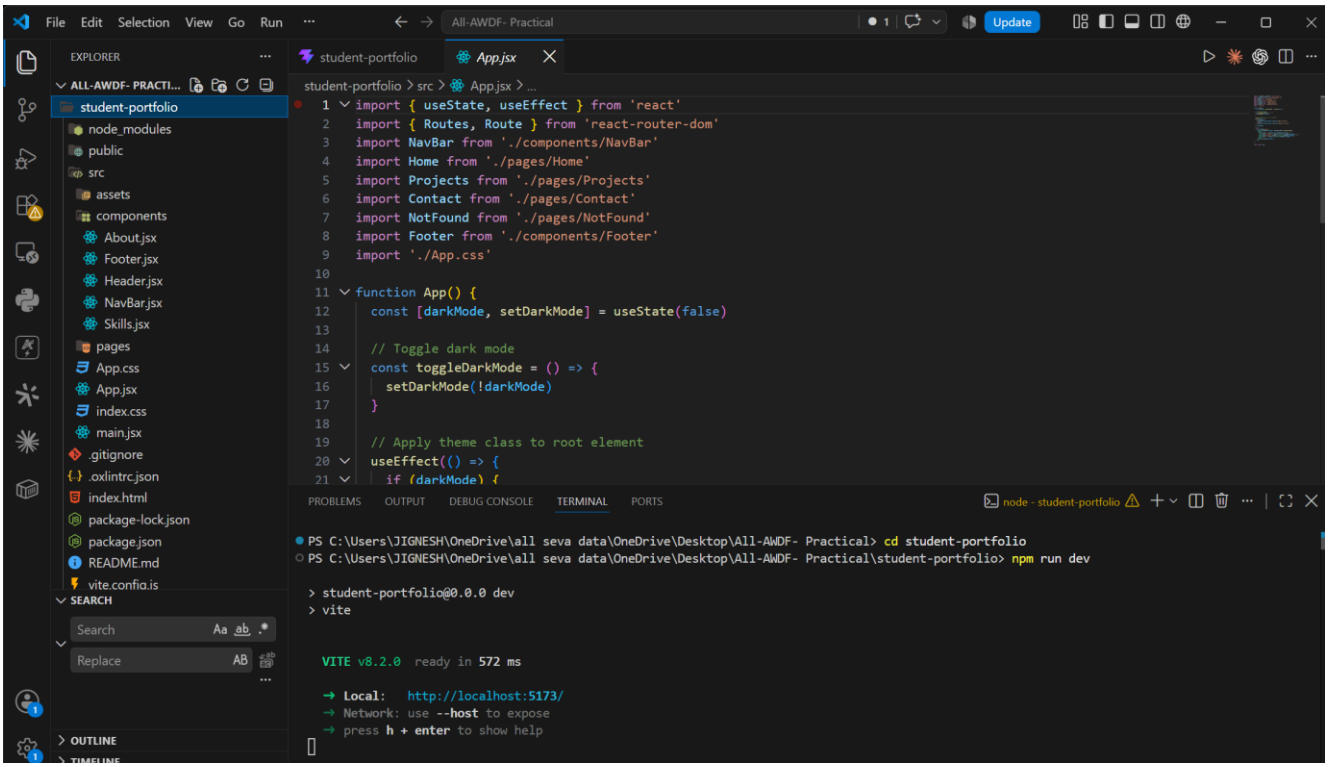
```
function Skills({ skillList }) {
 return (
  <ul>
   {skillList.map((s) => <li key={s}>{s}</li>)}
  </ul>
 );
}
```

```
function Skills({ darkMode }) {
  const skillCategories = {
    "Languages": ["Python", "JavaScript", "C", "HTML5", "CSS3"],
    "Frontend": ["React", "JavaScript", "HTML5", "CSS3", "Responsive Design"],
    "Backend": ["Python", "FastAPI", "Django", "REST API Development"],
    "Database": ["MySQL", "SQLite", "PostgreSQL"],
    "Cloud & Tools": ["AWS S3", "Git", "GitHub", "VS Code", "PyCharm", "npm", "Vite"]
  }
```

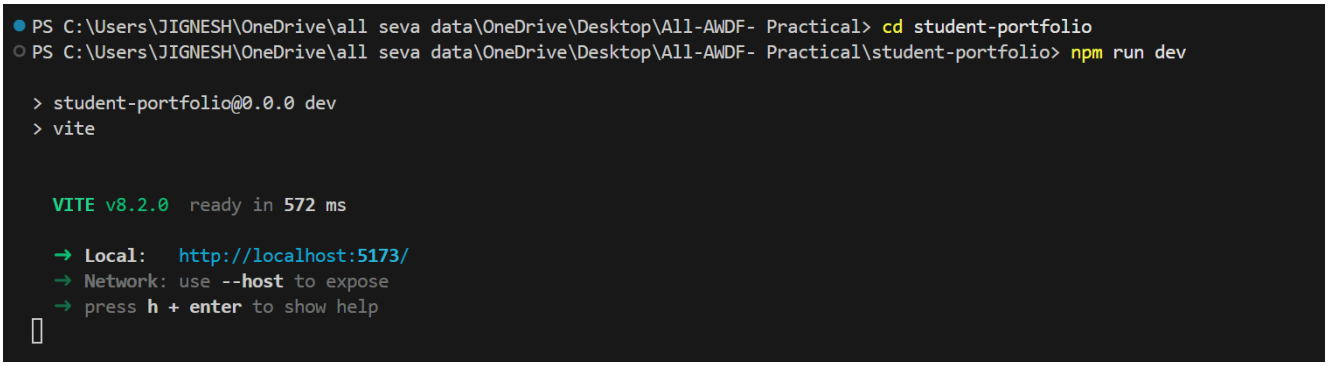
5. Import and compose all four components inside App.jsx in a single layout.

```
import Home from './pages/Home'
import Projects from './pages/Projects'
import Contact from './pages/Contact'
import NotFound from './pages/NotFound'
import Footer from './components/Footer'
import './App.css'
```

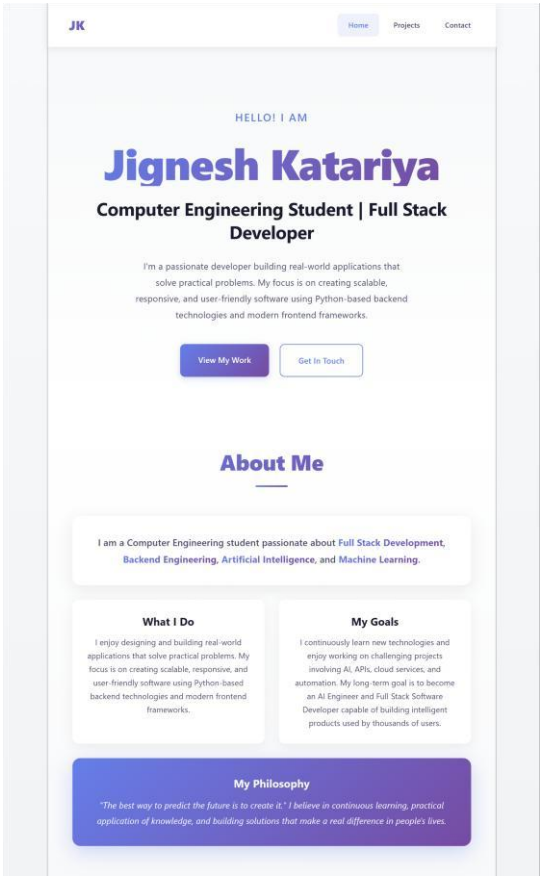
6. Pass a name prop to Header and a skillList prop to Skills from App.jsx.

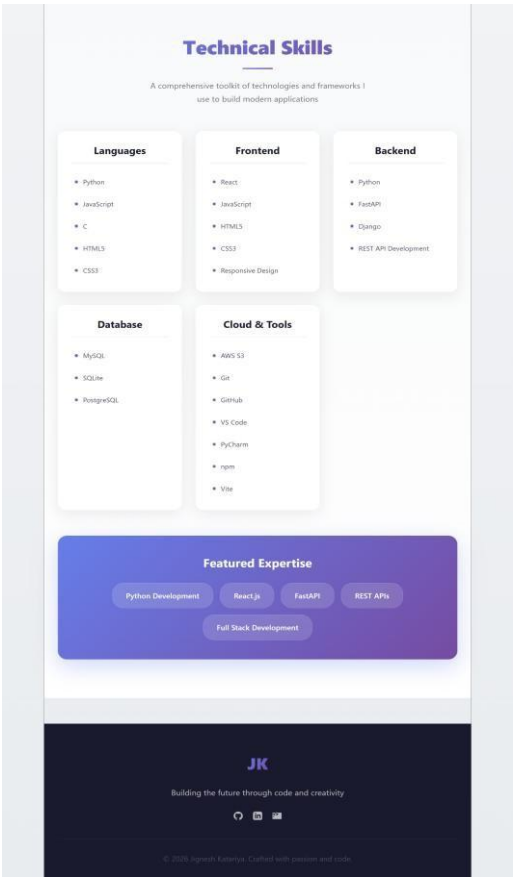


7. Run the app and verify all components render correctly without console errors.



Output





Key Questions / Analysis

1. **What is the role of each component in the overall UI structure?**

Each component represents a specific part of the UI, manages its own logic, and combines with other components to build the complete application. This makes the UI modular and easier to maintain.

2. **How does React re-render when props change?**

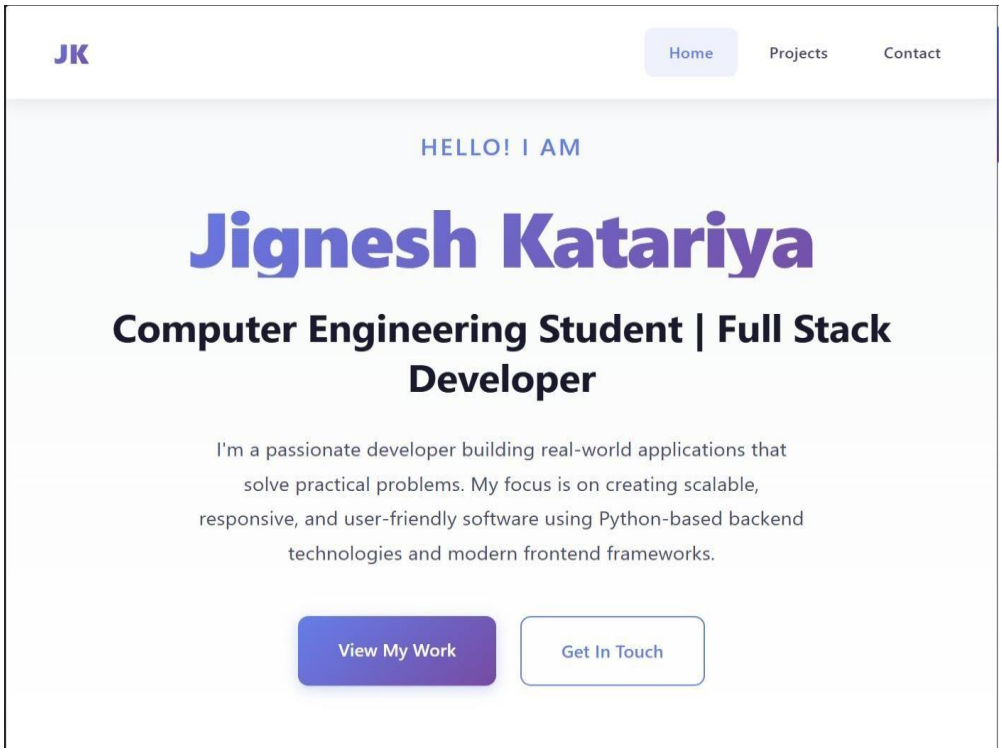
When a parent passes new props to a child component, React compares the changes using the Virtual DOM. If the props have changed, React re-renders the affected component and updates only the necessary parts of the real DOM.

3. **Why is component reusability important in large-scale applications?**

Component reusability reduces code duplication, improves consistency, simplifies maintenance, and speeds up development by allowing the same component to be used in multiple places.

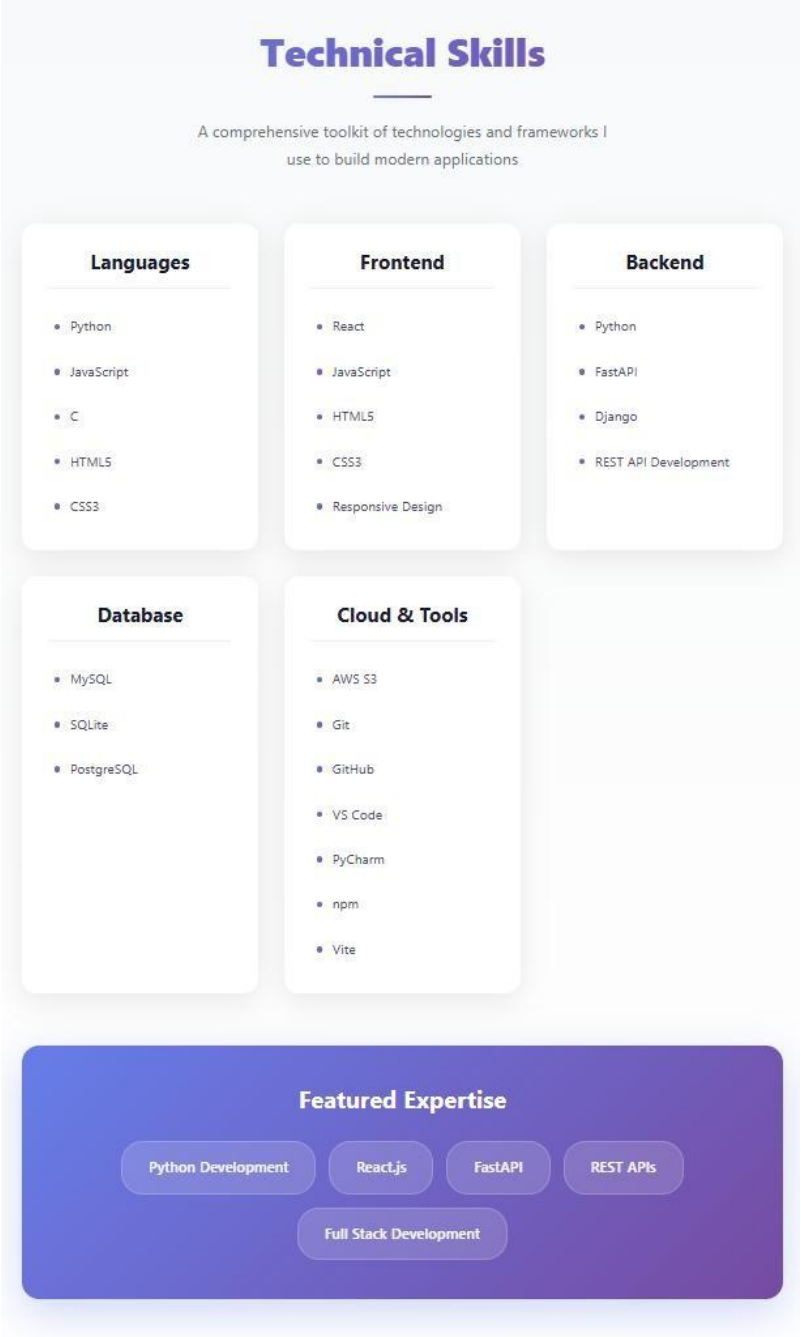
Supplementary Problems

1. **Add a NavBar component that highlights the active section.**



2. **Add a theme color prop to the Header and apply it as an inline style.**



	3. Pass an array of skills as a prop to the Skills component and render them dynamically. 
Key Skills Addressed	1. Setting up a React project using Vite 2. Designing and structuring reusable functional components 3. Passing data between components using props 4. Composing a single-page layout from multiple components
Learning Outcome	Students will be able to set up a React development environment, create reusable components, and compose them into a working UI using props for data flow.